**Hands on 4**

# Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

- JSR 338 Specification for persisting, reading and managing data from Java objects
- Does not contain concrete implementation of the specification
- Hibernate is one of the implementation of JPA

Hibernate

- ORM Tool that implements JPA

Spring Data JPA

- Does not have JPA implementation, but reduces boiler plate code
- This is another level of abstraction over JPA implementation provider like Hibernate
- Manages transactions

**The below code compares between Hibernate and Spring Data JPA**
**Hibernate**

```
   /* Method to CREATE an employee in the database */
   public Integer addEmployee(Employee employee){
      Session session = factory.openSession();
      Transaction tx = null;
      Integer employeeID = null;

      try {
         tx = session.beginTransaction();
         employeeID = (Integer) session.save(employee);
         tx.commit();
      } catch (HibernateException e) {
         if (tx != null) tx.rollback();
         e.printStackTrace();
      } finally {
         session.close();
      }
      return employeeID;
   }
```

**Spring Data JPA**

EmployeeRespository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
```

```
}
```

EmployeeService.java

```
    @Autowire
    private EmployeeRepository employeeRepository;


    @Transactional
    public void addEmployee(Employee employee) {
        employeeRepository.save(employee);
    }
```

## 1. JPA (Java Persistence API)

JPA is just a **specification (rulebook)**, not a tool.

- Defined under **JSR 338**
- Used for managing relational data in Java applications
- It tells *what to do*, but not *how to do it*
- Does NOT contain implementation code

Think of JPA as:

A blueprint for ORM (Object Relational Mapping)

Example:

- Entities, annotations like @Entity, @Id, etc.

# 2. Hibernate

Hibernate is a **JPA implementation + ORM tool**

- Implements JPA specifications
- Handles actual database operations
- Provides Session, Transaction management
- More powerful features than JPA alone

Think of Hibernate as:

The engine that executes JPA rules

### Example (Hibernate code flow)

- Open Session
- Begin Transaction
- Save object
- Commit / rollback
- Close session

More boilerplate code
Full control over DB operations

## 3. Spring Data JPA

Spring Data JPA is a **high-level abstraction over JPA**

- Uses JPA (usually Hibernate internally)
- Reduces boilerplate code drastically
- Provides ready-made methods like:
  - save()
  - findById()
  - delete()
- Handles transactions automatically using @Transactional

Think of it as:

A simplified layer over JPA + Hibernate

## Key Difference Table

| Feature | JPA | Hibernate | Spring Data JPA |
|---|---|---|---|
| Type | Specification | ORM Tool / Implementation | Abstraction layer |
| Works alone? | No | Yes | No (needs JPA provider) |
| Boilerplate code | Medium | High | Very low |
| Transaction handling | No | Manual | Automatic |
| CRUD operations | No | Yes | Yes (built-in methods) |
| Complexity | Medium | High | Low |

## Code Difference Summary

### Hibernate Approach

- Manually manage session
- Manually handle transactions

More control
More code

### Spring Data JPA Approach

employeeRepository.save(employee);

No session handling
No transaction handling (handled by Spring)
Very less code

**Java Persistence API (JPA)** is a specification that defines a standard way to map Java objects to database tables. It provides guidelines for performing CRUD (Create, Read, Update, Delete) operations but does not include any actual implementation. Since it is only a specification, we need an implementation like Hibernate to use it in a real application.

**Hibernate** is one of the most popular implementations of JPA. It is an Object-Relational Mapping (ORM) framework that converts Java objects into database records and vice versa. Hibernate manages SQL generation, object mapping, caching, and transaction handling. Although it provides many advanced features and greater control over database operations, developers have to write additional code to create sessions, begin transactions, commit changes, and handle exceptions.

**Spring Data JPA** is built on top of JPA and further simplifies database programming. It does not replace Hibernate but works with a JPA implementation such as Hibernate. Instead of writing repetitive DAO classes and SQL queries for common operations, developers can simply create a repository interface by extending JpaRepository. Spring Data JPA automatically provides methods such as save(), findAll(), findById(), and delete(). It also integrates well with Spring's transaction management using the @Transactional annotation, making the code cleaner and easier to maintain.

From the code comparison, I observed that Hibernate requires developers to manually open sessions, begin transactions, save entities, commit transactions, and close sessions. In contrast, Spring Data JPA reduces this entire process to a single method call such as employeeRepository.save(employee). This greatly reduces boilerplate code, improves readability, and increases developer productivity.

Overall, I learned that these technologies work together rather than compete with each other. **JPA defines the standard**, **Hibernate implements that standard**, and **Spring Data JPA provides a simpler and more efficient way to use JPA implementations**. In most modern Spring Boot applications, developers commonly use Spring Data JPA with Hibernate because it minimizes coding effort while still providing powerful database access features

This hands-on helped me understand the relationship between JPA, Hibernate, and Spring Data JPA. I learned that JPA is only a specification, Hibernate is the implementation of that specification, and Spring Data JPA is an abstraction that simplifies data access by reducing boilerplate code. Using Spring Data JPA with Hibernate makes Java applications easier to develop, maintain, and scale, which is why this combination is widely used in enterprise applications today.